REVISITING LIFETIME-AWARE VM ALLOCATION: RECONSTRUCTION, DIAGNOSIS AND PRIOR-FUSED LIFETIME PREDICTION

CE7490 Course Project · 26S1 Group 5

Zhang Qi (G2510784F)
qi015@e.ntu.edu.sg

Zhang Saining (G2511061H)
saining002@e.ntu.edu.sg

Zheng Yitao (G2608664B)
YZHENG056@e.ntu.edu.sg

ABSTRACT

Lifetime-aware VM allocation can improve consolidation, but inaccurate lifetime estimates may reverse its benefit. We reconstruct LAVA and confidence-aware variants in an event-driven simulator with 180 hosts and 2,500 VMs per paired synthetic trace. On 100 fresh seeds, LAVA is 0.187 percentage points worse than Best-Fit at 25% prediction accuracy. Confidence routing does not resolve this failure: class confidence can be inverted, the gate passes nearly all requests under controlled noise, and host isolation incurs a packing cost. We instead fuse each noisy prediction with a profile-level lifetime prior, handling censoring and conditioning on uptime without changing the scheduling rules. In a pre-registered evaluation with Holm correction, the fused confidence-aware variant outperforms LAVA at 3 of 4 tested accuracy levels. Post-hoc analysis suggests that the gain comes mainly from the estimate rather than the gate. Robustness experiments identify underestimated prior lifetimes as the main observed failure mode. The findings apply to the reconstructed synthetic environment and do not establish production gains.

1 INTRODUCTION

A VM scheduler must place requests that consume several resources at once. A feasible placement can still leave CPU and memory in incompatible proportions and strand capacity. Departure times add a temporal dimension: VMs that fit equally well at arrival may keep a host occupied for very different periods. Empty hosts matter for maintenance, large requests and energy management, although this simulator measures empty capacity rather than those outcomes directly.

LAVA [1] combines learned lifetime distributions, re-prediction with uptime and host lifecycle adaptation. The production system relies on private traces and integration with Borg [2]. This project studies the public scheduling ideas in a transparent synthetic environment, where policies can be compared on identical workloads and identical prediction realizations.

Resource Central showed that historical VM behavior can support resource prediction [3]. Online VM allocation with lifetime and load predictions [4] and lifetime-alignment placement [5] are direct precursors; our LA-Binary baseline follows the comparison in LAVA [1]. Protean illustrates the architecture and operational constraints of a large-scale VM allocation service [6]. LAVA adds distributions, re-prediction and host lifecycle states [1]. Our prior-fused predictor combines Bayesian updating [7] with right-censored survival information [8] to refine the noisy predictions consumed by the scheduler.

The original course implementation and traces are unavailable. We therefore reconstruct the public scheduling mechanisms from the archived course report and presentation in *reference/*, declaring inferred parameters explicitly rather than fitting them to historical outcomes. This scope supports paired comparisons within a synthetic environment; it does not recover the lost commit or reproduce production performance.

This reconstruction asks whether LAVA's benefit tracks lifetime-prediction quality (RQ1), whether routing uncertain arrivals to Best-Fit protects the scheduler from unreliable predictions (RQ2), and whether improving the lifetime estimate itself is more effective than switching placement policies (RQ3). We compare policies on identical synthetic traces and prediction realizations, separating the scheduling rules from the predictor.

Contributions and findings. We provide an auditable reconstruction, diagnose confidence-routing failure, and evaluate a predictor-level remedy. Unreliable predictions can make LAVA worse than Best-Fit. Inverted confidence, an unselective gate and host isolation help explain why routing does not help here. Prior fusion improves on LAVA in 3 of 4 pre-registered tests on 100 unused seeds, with scheduling rules unchanged. Underestimated prior lifetimes are the main observed failure mode.

2 SCHEDULING ALGORITHMS

Table 2.1. Policies compared. "*-Bayes" denotes the four variants that only swap in the prior-fused predictor.

Policy	Placement and prediction use
Best-Fit	Minimize squared residual CPU plus squared residual memory *Prediction:* None for placement
LA-Binary	Prefer non-empty hosts with the same initial short/long label, then other non-empty hosts, then empty hosts; Best-Fit within each group *Prediction:* One-shot label at 2 hours
NILAS	Quantized extension of the predicted latest host exit, then Best-Fit *Prediction:* Re-predicts live residents
LAVA	Lifecycle state and lifetime class, then NILAS and Best-Fit *Prediction:* Re-predicted remaining lifetime
VM-Level CA (VM-CA)	LAVA or Best-Fit, chosen per arrival *Prediction:* Class confidence ≥ threshold τ
Host-Level CA (Host-CA)	Same gate; eligible hosts restricted to the same mode or empty *Prediction:* Same gate as VM-CA
*-Bayes variants (new)	Identical rules to NILAS, LAVA, VM-CA or Host-CA *Prediction:* Prior-fused prediction (Section 8)

Prediction-driven resource management also includes Quasar, which infers resource and interference characteristics for QoS-aware allocation [9]. TetriSched uses estimated runtimes in adaptive plan-ahead scheduling [10], while 3Sigma explicitly represents runtime uncertainty with distributions [11]. LAVA also draws on lifetime re-estimation in learning-based memory allocation for C++ servers [12]. These systems provide context for using predictions in scheduling; the experiments here isolate VM lifetimes and two-resource host packing.

LAVA classes use remaining-lifetime boundaries at 1, 10 and 100 hours, supplemented from the public paper [1]. A newly occupied host is *open*. When CPU or memory reaches 90%, it becomes *recycling* and records its current VMs as the residual cohort. Placement prefers recycling hosts in a higher class than the arriving VM (closest class first), then open hosts in the same class, then other non-empty hosts and finally empty hosts. When the residual cohort exits, a non-empty recycling host drops one class; deadline expiry raises the class and refreshes the cohort.

VM-CA routes each arrival by its confidence. Because hosts are shared, one host can receive both LAVA and Best-Fit decisions; we call this *mode mixing*. Host-CA assigns a mode at a host's first placement and keeps it until the host empties. It considers same-mode and empty hosts first and allows a cross-mode placement only when no such host can admit the request (capacity fallback). No fallback occurred in any recorded suite.

The four *-Bayes policies run exactly the scheduling code of their base policy. Only the predictor object differs, so any difference in outcome is attributable to the lifetime estimate and its confidence. Under the default predictor and the Oracle there is no noisy value to fuse, and the variants reproduce their base policies bit for bit.

3 WORKLOAD MODEL AND PROVENANCE

Each host has normalized CPU and memory capacity of one. Every trace contains 2,500 VM requests at a documented mean arrival rate of eight per simulated hour. The reconstruction uses independent exponential interarrival times; the surviving report gives the mean rate but not the arrival process, so Poisson arrivals are an explicit assumption (Table 3.1).

Table 3.1. Simulator parameters and their provenance.

Parameter	Value	Status
Hosts / requests	180 / 2,500 per trace	Recovered
Mean arrival rate	8 requests/hour	Recovered
Interarrival distribution	Independent exponential	Inferred
Profile probabilities	Interactive 0.45, batch 0.35, service 0.20	Inferred
Prediction cap	336 hours (14 days)	Recovered
Recycling threshold	0.90 CPU or memory	Recovered
LAVA class boundaries	1, 10, 100 hours	Public paper supplement
Best-Fit score	Residual CPU squared + residual memory squared	Inferred

VM lifetimes follow a two-exponential mixture. For each profile, p is the probability of the short component and a, b are the component means (Table 3.2). The profile names and the two-component family are recovered; all numeric values are declared synthetic defaults.

Table 3.2. Lifetime mixture of each VM profile.

Profile	p (short component)	a (hours)	b (hours)
Interactive	0.9	0.3	8
Batch	0.65	2	30
Service	0.3	4	120

The inferred (CPU, memory) shapes are (0.125, 0.125), (0.25, 0.125), (0.125, 0.25), (0.5, 0.25) and (0.25, 0.5) with probabilities 0.35, 0.20, 0.20, 0.125 and 0.125. Shape, profile and component are sampled independently. Lifetimes are not truncated; only predictions receive the 14-day cap.

These values were declared once and never fitted to the earlier report or to any result. A different plausible mixture could change relative policy performance; Section 10 varies predictor-side assumptions and the load, but not the generating mixture itself.

4 LIFETIME PREDICTION

Conditioning on observed uptime uses survival information to update the remaining-lifetime distribution [8, 13]. The two-exponential mixture and its parameters are explicit synthetic assumptions (Table 3.2), rather than a fit to a production trace.

Conditional lifetime model

Let T follow a two-exponential mixture with short-component probability p and means a and b. At uptime u the short-component weight and the expected remaining lifetime are

$$w(u) = p \cdot e^{-u/a} / [\, p \cdot e^{-u/a} + (1-p) \cdot e^{-u/b} \,],$$

$$E[T - u \mid T > u] = w(u) \cdot a + (1 - w(u)) \cdot b.$$

The weights are evaluated in log space to avoid underflow. The *default predictor* returns this conditional mean, capped at 336 hours. Its confidence is the conditional probability that the remaining lifetime falls in the LAVA class that contains the mean.

Controlled-noise protocol

Following the controlled-noise protocol of [1, Appendix G.1], the harness assigns each VM to the *correct bucket* with probability equal to the requested accuracy and to the *incorrect bucket* otherwise. It applies Gaussian noise in log10 space with $\sigma = 0.001$ (correct) or $\sigma = 3$ (incorrect) to the true total lifetime and caps the result at 14 days. All policies with the same seed and setting see the same noisy values; the bucket label never reaches the scheduler.

The original controlled predictor freezes the noisy value. Before that value is reached, remaining lifetime is the value minus uptime; afterwards it switches to min(336, max(0.1, uptime)) hours. Its confidence is a posterior class mass under a log-uniform prior and the global error model (accuracy and the two σ values).

Oracle predictions use the actual remaining lifetime with confidence one. This full-information reference measures how much the same scheduling rules can gain with perfect lifetimes. It is not an upper bound for every policy (Section 11).

The default confidence is the probability of the class that contains the mixture *mean*. For bimodal mixtures the mean often lies in a low-probability class (Section 7). The controlled predictor trusts every noisy value, including errors that span several orders of magnitude (Section 7). When a VM outlives its noisy value, its predicted remaining lifetime jumps to min(336, max(0.1, uptime)), and a value at the 336-hour cap is treated as an exact observation rather than as censored.

Section 7 shows how the first two properties affect confidence-aware routing. Section 8 addresses the second and third directly.

5 EXPERIMENTAL PROTOCOL

Metrics and statistical inference

The primary metric is 100 times the time integral of the number of empty hosts, divided by the number of hosts and the interval from the first arrival to the final exit. It includes the full drain tail. Results are paired differences in percentage points (pp) relative to the same-seed baseline. A secondary diagnostic integrates only the arrival window (first to last arrival) to separate packing during the busy period from consolidation during the drain tail.

Scale matters for reading the results. Best-Fit leaves 89.6% of host-time empty on average (86.3–93.4% across test seeds), because the offered load is low and the drain tail is long. The arrival window lasts about 312 hours of a 985-hour horizon, so about 68% of the evaluation horizon is the drain tail. With 180 hosts, 1 pp equals 1.8 hosts on average over the horizon. The headline gain of +0.224 pp is therefore about 0.40 of a host, or 398 empty host-hours per trace (2.2% of Best-Fit's busy host-hours).

The offered load is about 20% at steady state (8 VMs per hour × 21.65 h mean lifetime × 0.2125 CPU, over 180 hosts). Because the 2,500 arrivals end before steady state is fully reached, realized CPU utilization is about 14% during the arrival window and 7% over the full horizon.

For each seed the analysis subtracts the baseline metric from the alternative and forms a two-sided Student-t 95% interval with $n-1$ degrees of freedom, applied to the per-seed differences [14]. It refuses comparisons with missing pairs, failed admissions or unequal accepted counts. Intervals in all tables are unadjusted. Only the eight registered tests of Section 9 also carry Holm–Bonferroni adjusted p-values [15]; all other comparisons are descriptive.

Table 5.1. Experiment suites. Controlled sweep accuracies are 25%, 50%, 75%, 80%, 85%, 90%, 95% and 100%; later suites use 25%, 50%, 75%, 85% and 100%. *Added after review; descriptive.

Suite / seeds	Settings and policies	Runs
Smoke 0–1	3 predictor settings, 6 policies	36
Initial sweep 0–19	8 accuracies + default + Oracle; Best-Fit, LAVA, VM-CA	600
Validation 200–205	Default predictor; 7 thresholds; 4 policies	168
Held-out 20–39	5 accuracies + default + Oracle; 6 policies	840
Improvement test 1000–1099	5 accuracies + default + Oracle; 10 policies (pre-registered)	7,000
Legacy rerun 20–39	Same 7 settings; 10 policies	1,400
Threshold sensitivity 200–219	4 accuracies; 7 thresholds; 4 policies	2,240
Robustness 2000–2049	4 accuracies; 7 predictor or load variants; 7 policies	9,800
Robustness control* 2000–2049	4 accuracies; unmodified protocol; 7 policies	1,400
Load ×2* 2000–2049	4 accuracies; arrival rate 16/h, 5,000 VMs; 7 policies	1,400

Registration and verification

The protocol distinguishes planned tests from later exploratory analyses, following the preregistration principle [16]. Development of the new method used only seeds 0–19. The hypotheses, test seeds, frozen threshold, correction and decision rule were written to *configs/improvement_protocol.json*, with SHA256 hashes of all simulator sources, at 20:10 (UTC+8) on 28 October 2026. This was before any prior-fused run on the legacy, threshold, test or robustness seeds. The registration is a local file with a self-reported timestamp, not an external registry. No simulator source changed afterwards, and all six improvement-suite manifests record the registered hashes. The development exploration script was packaged after registration; it reproduces exactly the development numbers recorded in the registration file. The two suites marked * in Table 5.1 were added after review.

All 24,884 runs accept every request with zero scheduling failures and no oversubscription. Twenty-nine unit tests pass, covering event ordering, time integration, mixture conditioning, resource bounds, host isolation, capacity fallback, Oracle equivalence, the no-truth-access boundary of both predictors, censoring, and the paired t-test and Holm adjustment. Re-simulating the 1,644 stored reconstruction runs reproduces every field bit for bit on macOS; on Linux all integer decision counts match and floating-point values differ only in the last digits (relative difference $\leq 3.3\times10^{-16}$). In the other direction, an evenly spaced sample of 300 improvement runs computed on the Linux server, re-simulated on macOS, matches every integer field, with floating-point differences of at most 5.2×10^{-16}. *scripts/verify_reproduction.py* performs these checks and writes *results/verification_log.txt*.

The simulator is pure-Python CPU code and needs no GPU. A report-scale run takes about 0.1–0.3 s for the original policies and 0.7 s with the fused predictor on one Apple M5 core, and about 0.9 s on average on one core of the server used below. The 23,240 improvement runs used 64 low-priority processes on a 192-thread Xeon server and finished in about 6 minutes of wall time; the server's H20 GPUs were not used.

6 PREDICTION ACCURACY SENSITIVITY

Figure 6.1 shows LAVA minus Best-Fit across controlled accuracy on the initial split.

The first tested level with a positive paired interval is 75%, and the estimated gain keeps increasing through the high-accuracy regime. This answers RQ1 within the reconstructed workload. The significance point is a property of the tested grid, the workload and the implementation, not a universal threshold; the scan is descriptive and not corrected for multiplicity.

The 100 fresh test seeds of Section 9 confirm the shape with tighter intervals: LAVA minus Best-Fit is −0.187 [−0.240, −0.135] pp at 25%, −0.025 [−0.077, +0.027] at 50%, +0.218 [+0.162, +0.274] at 75%, +0.309 [+0.250, +0.369] at 85% and +0.507 [+0.445, +0.569] at 100%; the Oracle reaches +0.552 [+0.491, +0.613]. **At 25% accuracy LAVA is significantly worse than Best-Fit**: trusting unreliable predictions harms packing. This is the regime that RQ2 and RQ3 target.

Under the default predictor, LAVA's small gain on the initial split (+0.124 [+0.005, +0.242] pp) does not replicate on the held-out seeds (+0.001 [−0.104, +0.105]) or on the fresh seeds (−0.043 [−0.098, +0.012]). For comparison, the initial-split Oracle gain is +0.575 [+0.433, +0.718]; the full sweep values are archived.

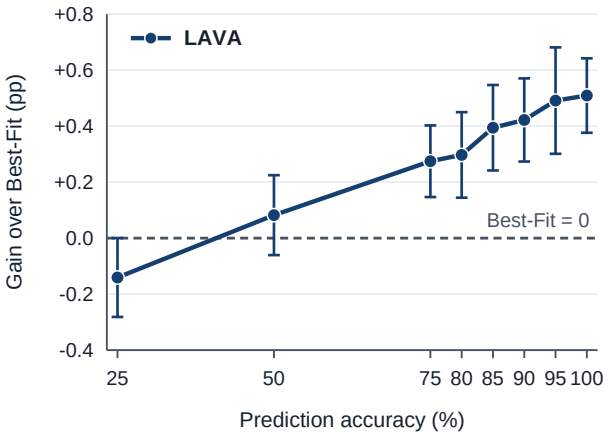


Figure 6.1. LAVA minus Best-Fit across controlled accuracy, initial split (seeds 0–19). Percentage points; paired means with 95% intervals, n = 20 per point; the accuracy axis is to scale.

Learning-augmented online algorithms distinguish gains from accurate advice from protection against bad predictions [17]. Error-dependent bounds for dynamic bin packing with predictions formalize that distinction [18]. The present study reports paired empirical comparisons in a two-resource simulator, without claiming competitive-ratio guarantees.

7 CONFIDENCE-AWARE ROUTING

7.1 Outcomes and placement behavior

Table 7.1 reports the held-out comparison (seeds 20–39) relative to Best-Fit, with the threshold fixed at the report value 0.3. At 50% accuracy and above, both confidence-aware variants follow LAVA exactly, because the gate passes every arrival.

Table 7.1. Held-out seeds 20–39, pp versus Best-Fit, mean [95% CI].

Setting	LAVA	VM-CA	Host-CA
25%	−0.137 [−0.247, −0.026]	−0.145 [−0.261, −0.030]	−0.115 [−0.234, +0.004]
50%	+0.069 [−0.022, +0.159]	+0.069 [−0.022, +0.159]	+0.069 [−0.022, +0.159]
75%	+0.215 [+0.096, +0.333]	+0.215 [+0.096, +0.333]	+0.215 [+0.096, +0.333]
85%	+0.399 [+0.281, +0.517]	+0.399 [+0.281, +0.517]	+0.399 [+0.281, +0.517]
100%	+0.598 [+0.490, +0.706]	+0.598 [+0.490, +0.706]	+0.598 [+0.490, +0.706]
Synthetic	+0.001 [−0.104, +0.105]	+0.114 [+0.007, +0.220]	−0.045 [−0.138, +0.049]
Oracle	+0.661 [+0.549, +0.774]	+0.661 [+0.549, +0.774]	+0.661 [+0.549, +0.774]

Under the default predictor, Host-CA does not improve on Best-Fit on the held-out seeds and is significantly worse than VM-CA there (Table 7.2). On 100 fresh seeds it is significantly worse than Best-Fit (−0.090 [−0.147, −0.034] pp). The small held-out gain of VM-CA does not replicate (−0.015 [−0.070, +0.041] pp). For reference, the surviving report gave Host-CA minus Best-Fit as +0.241 [+0.125, +0.357] pp; this historical value is not reproduced.

Table 7.2. Paired comparisons of Host-CA under the default predictor.

Default predictor, Host-CA minus …	Held-out seeds 20–39 (n = 20)	Fresh seeds 1000–1099 (n = 100)
Best-Fit	−0.045 [−0.138, +0.049]	−0.090 [−0.147, −0.034]
LAVA	−0.046 [−0.169, +0.078]	−0.047 [−0.097, +0.002]
VM-CA	−0.158 [−0.303, −0.013]	−0.075 [−0.130, −0.021]

Host isolation removes exactly what it was designed to remove. Both variants receive the same gate sequence, and Host-CA reduces both cross-mode counts to zero without any capacity fallback (Table 7.3). The missing gain is therefore not an implementation defect of the invariant.

Table 7.3. Mechanism counts under the default predictor, held-out seeds.

Diagnostic, mean per trace (seeds 20–39)	VM-CA	Host-CA
Gate changes per 1,000 arrivals	318.64	318.64
Best-Fit decisions placed on LAVA-mode hosts	607.75	0
LAVA decisions placed on Best-Fit-mode hosts	319.40	0
Capacity fallbacks	0	0

7.2 Why routing fails

Inverted confidence. Under the default predictor every request arrives with uptime zero, so its confidence is a fixed number per profile (Figure 7.1). The confidence measures the probability of the class that contains the mixture mean, which for bimodal mixtures is often an unlikely class. Interactive VMs are the most predictable profile, yet they receive the lowest confidence; service VMs are the least predictable, yet they receive the highest. At $\tau = 0.3$ only service VMs pass the gate, so Host-CA reduces to isolating service VMs on their own hosts.

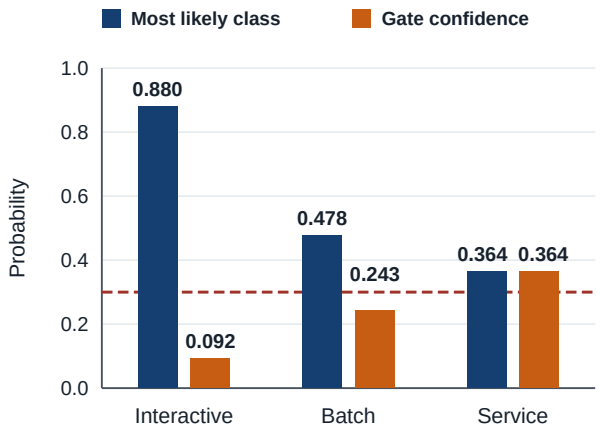


Figure 7.1. Predictability versus gate confidence per profile. The gate confidence is lowest for the most predictable profile and highest for the least predictable one.

Fixing the inversion alone is not enough. On the development seeds, routing by the most likely class and its probability did not beat LAVA at any tested threshold (−0.129 to −0.043 pp versus LAVA for the VM-level and host-level variants at $\tau = 0.4$, 0.5 and 0.9; Section 8). The inversion is a contributing factor, not a sufficient explanation.

Calibration concerns agreement between predicted confidence and empirical correctness [19]. The class probability here follows the specified mixture; the observed failure concerns the mean-selected class as a decision summary. A valid probability for that event need not rank VM predictability well or provide a useful routing order for packing.

Unselective gating. At arrival, the correct-bucket error of the original controlled predictor lies almost entirely inside the predicted class, so the confidence is at least about the global accuracy. At 50% accuracy and above this bound alone exceeds $\tau = 0.3$; at 25% almost every request still passes (Table 7.4). Confidence-aware routing therefore cannot separate the errors that actually harm packing.

Table 7.4. Share of arrivals routed to LAVA at $\tau = 0.3$, fresh seeds (n = 100); fused confidence (Section 8) for comparison.

Accuracy	VM-CA (original confidence)	VM-CA-Bayes (fused confidence)
25%	99.96%	75.46%
50%	100.00%	78.74%
75%	100.00%	80.58%
85%	100.00%	84.77%
100%	100.00%	100.00%
Synthetic	20.09%	20.09%

Packing cost. Host-CA restricts every request to hosts of its own mode or to empty hosts, so requests often start new hosts that could otherwise have been shared. During the arrival window Host-CA has 0.99 pp fewer empty hosts than Best-Fit; over the full horizon the deficit is 0.09 pp, so the drain tail recovers about 72% of the lost empty host-hours (Table 7.5). Combining isolation with the fused predictor makes it worse (Section 9).

Table 7.5. Arrival-window and full-horizon differences, fresh seeds (n = 100), pp.

Comparison	Setting	Arrival window	Full horizon (primary)
Host-CA − Best-Fit	Synthetic	−0.992 [−1.028, −0.957]	−0.090 [−0.147, −0.034]
Host-CA − VM-CA	Synthetic	−0.857 [−0.894, −0.821]	−0.075 [−0.130, −0.021]
Host-CA-Bayes − VM-CA-Bayes	50%	−0.699 [−0.734, −0.664]	−0.284 [−0.342, −0.226]

7.3 Threshold implications

On validation seeds 200–205 the reconstructed objective (mean Host-CA improvement over LAVA and VM-CA) selects $\tau = 0.5$ by smallest-threshold tie-breaking: $\tau = 0.5$, 0.7, 0.8, 0.9 tie at −0.085 pp, because at these thresholds no profile passes the gate and Host-CA equals Best-Fit. Every candidate score is negative, so changing the threshold cannot rescue host-level routing; the held-out comparisons keep the frozen report value 0.3.

Confidence-aware routing does not help here, and threshold tuning did not change that. The confidence signal is inverted under the default predictor and uninformative under controlled noise, and host isolation has a measurable packing cost. Mode mixing is real, but removing it does not produce a gain. The negative result points at the predictor, which RQ3 changes.

8 Prior-fused lifetime prediction

The diagnosis suggests repairing the lifetime estimate rather than switching policies. The scheduler already knows a profile-level lifetime distribution, because the default predictor is exactly that model. Under controlled noise it also receives a per-VM noisy value whose error model is globally known. Prior fusion combines both sources with Bayes' rule.

Posterior construction

The update combines a prior and an observation likelihood by Bayes' rule [7]. Let T denote the VM's total lifetime and o its noisy lifetime observation. The prior is the VM's profile mixture, the same two-exponential model used by the default predictor. The observation model is $\log10\ o = \log10\ T + \varepsilon$ with $\varepsilon \sim \text{acc}\cdot N(0,\ 0.001^2) + (1-\text{acc})\cdot N(0,\ 3^2)$. The correct-bucket component is much narrower than the numerical grid, so its weight is the global accuracy times the prior density of $\log10\ T$ at $\log10\ o$. It is represented by nine atoms one σ apart over $\pm 4\sigma$.

A value at the 336-hour cap contributes $P(\log10\ T + \varepsilon \geq \log10\ 336)$, not a point likelihood [8]. Numerically, we use 425 weighted atoms on a log10 grid from 10^{-4} to $10^{4.5}$ hours (50 per decade), plus the correct-bucket atoms. Suffix sums make each re-prediction O(log n).

We keep the atoms above the current age. An overdue VM therefore falls back smoothly to the incorrect-bucket component, which removes the jump to max(0.1, uptime). If no atom remains, the predictor falls back to the profile prediction. The interface returns the posterior mean remaining lifetime (capped at 336 hours) and the posterior mass of its LAVA class. This is the same interface as the original predictor, so the NILAS, LAVA and CA rules are unchanged.

Information boundary and examples

The fused predictor uses the noisy value, the global accuracy, the error spreads, the cap and the profile. These are the quantities the original controlled confidence uses, plus the profile model that the default predictor already uses. That prior is the generating distribution; Section 10 tests misspecified priors. The predictor never sees the bucket label or the true lifetime. A unit test rebuilds it on records whose true durations are multiplied by 50, keeps the same noisy values and verifies identical predictions.

For an interactive VM at 25% accuracy, the 300-hour noisy value is implausible under its prior. The fused estimate is 1.52 h, close to the profile mean of 1.07 h; the original predictor returns 300 h. For a service VM at 50% accuracy, a plausible 2-hour observation moves the posterior mean to 27.0 h, between the raw value of 2 h and the prior mean of 85.2 h.

Censoring matters when a service VM at 100% controlled accuracy has a capped noisy prediction at the 336-hour cap. At uptime 300 h, the original predictor expects 36 h remaining and the fused predictor 151 h, because the uncapped noisy observation is at least 336 h, rather than an exact departure time. At uptime 400 h, the original estimate jumps to 336 h with confidence 0.0, while the fused estimate is 118 h.

Development and frozen choices

Two other ideas were explored on development seeds 0–19 and rejected before registration. An expected host-exit extension score computed from the full mixture gave no gain under the default predictor (LAVA variant −0.194 [−0.312, −0.076] pp versus LAVA). Routing by the most likely class and its probability did not beat LAVA at any tested threshold (−0.129 to −0.043 pp). On these seeds the confidence gate also appeared to add value on top of fusion: VM-CA-Bayes minus LAVA was +0.316, +0.187 and +0.134 pp at 25%, 50% and 75%, against +0.249, +0.100 and +0.049 pp for LAVA-Bayes. This is why VM-CA-Bayes was registered as the primary variant; Section 9 shows the gate's advantage did not replicate. The exploration script and its outputs are part of the artifact (scripts/explore_dev.py).

9 PRE-REGISTERED EVALUATION

The registered family compares VM-CA-Bayes against LAVA and against Best-Fit at 25%, 50%, 75% and 85% accuracy on fresh seeds 1000–1099 (n = 100). VM-CA-Bayes was registered because the project studies confidence-aware scheduling and it was the strongest variant on development seeds; 100% accuracy was excluded because the two predictors then differ only in overdue handling. The threshold stays at the frozen report value 0.3, and no parameter was tuned on any seed. An improvement is claimed only if the mean is positive and the Holm-adjusted p-value is below 0.05.

Table 9.1. Registered family (8 tests). Intervals are unadjusted; p-values are Holm-adjusted. 3 of 4 tests against LAVA and 3 of 4 against Best-Fit hold.

Acc.	VM-CA-Bayes − LAVA	VM-CA-Bayes − Best-Fit
25%	+0.224 [+0.174, +0.275] p_H: <0.001; yes	+0.037 [−0.012, +0.086] p_H: 0.14; no
50%	+0.147 [+0.088, +0.205] p_H: <0.001; yes	+0.122 [+0.064, +0.180] p_H: <0.001; yes
75%	+0.054 [−0.001, +0.109] p_H: 0.11; no	+0.272 [+0.217, +0.327] p_H: <0.001; yes
85%	+0.088 [+0.037, +0.139] p_H: 0.003; yes	+0.398 [+0.338, +0.457] p_H: <0.001; yes

Against LAVA, three of four tests hold. At 75% the interval includes zero (lower bound −0.001; Holm p = 0.11). Against Best-Fit, the tests at 75% and 85% would also hold for unmodified LAVA (+0.218 and +0.309 pp), so the evidence specific to fusion is the three LAVA tests and the Best-Fit test at 50%. At 25% the registered test against Best-Fit fails (+0.037 [−0.012, +0.086] pp). Secondary, unadjusted comparisons at 25% put NILAS-Bayes (+0.084 [+0.035, +0.134], p < 0.001) and

LAVA-Bayes (+0.055 [+0.006, +0.104], p = 0.03) above Best-Fit; they were not registered.

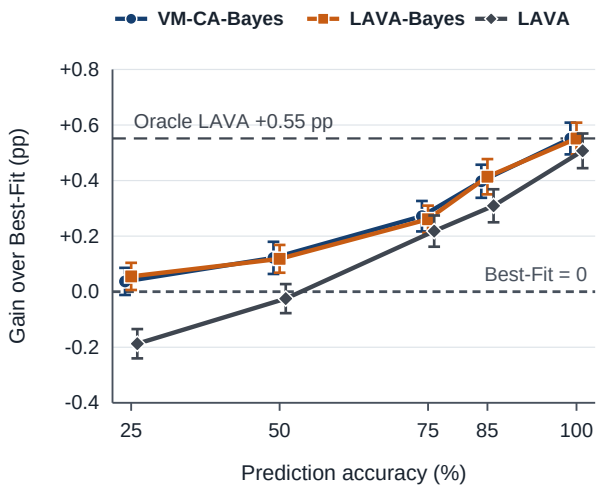


Figure 9.1. Gain over Best-Fit versus controlled accuracy, fresh seeds 1000–1099. VM-CA-Bayes is the registered variant; LAVA-Bayes (not registered) and LAVA are shown for reference. Paired means with unadjusted 95% intervals, n = 100; the accuracy axis is to scale. Markers are offset slightly for visibility.

Policy comparisons

Tables 9.2 and 9.3 list every policy on the test seeds. LA-Binary, omitted from the tables, is slightly below Best-Fit in every setting (−0.056 to −0.114 pp).

Table 9.2. Original policies, fresh seeds, pp versus Best-Fit. Shading marks intervals entirely above (green) or below (red) zero.

Setting	LAVA	NILAS	VM-CA	Host-CA
25%	−0.187 [−0.240, −0.135]	−0.055 [−0.108, −0.001]	−0.178 [−0.231, −0.125]	−0.180 [−0.236, −0.125]
50%	−0.025 [−0.077, +0.027]	+0.076 [+0.018, +0.134]	−0.025 [−0.077, +0.027]	−0.025 [−0.077, +0.027]
75%	+0.218 [+0.162, +0.274]	+0.218 [+0.158, +0.279]	+0.218 [+0.162, +0.274]	+0.218 [+0.162, +0.274]
85%	+0.309 [+0.250, +0.369]	+0.349 [+0.294, +0.403]	+0.309 [+0.250, +0.369]	+0.309 [+0.250, +0.369]
100%	+0.507 [+0.445, +0.569]	+0.476 [+0.420, +0.531]	+0.507 [+0.445, +0.569]	+0.507 [+0.445, +0.569]
Synthetic	−0.043 [−0.098, +0.012]	+0.049 [+0.002, +0.096]	−0.015 [−0.070, +0.041]	−0.090 [−0.147, −0.034]
Oracle	+0.552 [+0.491, +0.613]	+0.581 [+0.518, +0.644]	+0.552 [+0.491, +0.613]	+0.552 [+0.491, +0.613]

Table 9.3. Prior-fused variants on the same seeds; identical to their base policies under the default predictor and the Oracle.

Setting	NILAS-Bayes	LAVA-Bayes	VM-CA-Bayes	Host-CA-Bayes
25%	+0.084 [+0.035, +0.134]	+0.055 [+0.006, +0.104]	+0.037 [−0.012, +0.086]	−0.271 [−0.323, −0.220]
50%	+0.135 [+0.084, +0.187]	+0.118 [+0.068, +0.168]	+0.122 [+0.064, +0.180]	−0.162 [−0.221, −0.104]
75%	+0.324 [+0.268, +0.380]	+0.261 [+0.212, +0.310]	+0.272 [+0.217, +0.327]	+0.030 [−0.034, +0.093]
85%	+0.386 [+0.332, +0.441]	+0.414 [+0.350, +0.477]	+0.398 [+0.338, +0.457]	+0.199 [+0.139, +0.258]
100%	+0.599 [+0.538, +0.659]	+0.551 [+0.494, +0.609]	+0.551 [+0.494, +0.609]	+0.551 [+0.494, +0.609]
Synthetic	+0.049 [+0.002, +0.096]	−0.043 [−0.098, +0.012]	−0.015 [−0.070, +0.041]	−0.090 [−0.147, −0.034]
Oracle	+0.581 [+0.518, +0.644]	+0.552 [+0.491, +0.613]	+0.552 [+0.491, +0.613]	+0.552 [+0.491, +0.613]

Table 9.4 compares each fused variant with its own base policy, so the scheduling rules are identical within each column. Fusion helps NILAS, LAVA and VM-CA at most accuracies, most where predictions are least reliable. It makes Host-CA significantly worse at every accuracy from 25% to 85% (−0.091 to −0.189 pp), consistent with the more selective gate splitting the fleet further (Diagnosis 3).

Table 9.4. Fused variant minus its base policy, fresh seeds, pp.

Acc.	NILAS	LAVA	VM-CA	Host-CA
25%	+0.139 [+0.094, +0.185]	+0.242 [+0.182, +0.303]	+0.215 [+0.164, +0.266]	−0.091 [−0.146, −0.036]
50%	+0.059 [+0.005, +0.113]	+0.143 [+0.090, +0.197]	+0.147 [+0.088, +0.205]	−0.137 [−0.190, −0.085]
75%	+0.106 [+0.057, +0.154]	+0.043 [−0.010, +0.095]	+0.054 [−0.001, +0.109]	−0.189 [−0.246, −0.131]

Acc.	NILAS	LAVA	VM-CA	Host-CA
85%	+0.038 [−0.012, +0.088]	+0.105 [+0.054, +0.155]	+0.088 [+0.037, +0.139]	−0.111 [−0.161, −0.061]
100%	+0.123 [+0.082, +0.164]	+0.044 [−0.005, +0.093]	+0.044 [−0.005, +0.093]	+0.044 [−0.005, +0.093]

Post-hoc decomposition

The comparisons in Table 9.5 and the bucket analysis were not registered and are descriptive. Fusion improves on both of its ingredients (Table 9.5): the prior alone at every accuracy (second column, the same seeds under the default predictor) and the noisy predictions alone at every accuracy except 75%, where the interval includes zero (first column). Adding the confidence gate on top changes little: VM-CA-Bayes minus LAVA-Bayes lies between −0.018 and +0.011 pp, with intervals about ±0.05 pp wide, so a gate effect of that size cannot be excluded; the development-seed advantage of the gate did not replicate.

Table 9.5. Decomposition of the gain, fresh seeds, pp. The second column pairs each seed under controlled noise with the same seed under the default predictor; Best-Fit is identical in both.

Acc.	Fused − noisy	Fused − prior	Gate on top
25%	+0.242 [+0.182, +0.303]	+0.098 [+0.047, +0.149]	−0.018 [−0.066, +0.030]
50%	+0.143 [+0.090, +0.197]	+0.161 [+0.104, +0.218]	+0.003 [−0.050, +0.057]
75%	+0.043 [−0.010, +0.095]	+0.304 [+0.244, +0.363]	+0.011 [−0.043, +0.065]
85%	+0.105 [+0.054, +0.155]	+0.457 [+0.398, +0.515]	−0.016 [−0.066, +0.033]

The fused gate is more selective than the original one but discriminates only weakly between good and bad predictions (post-hoc bucket analysis). It routes 82–87% of correct-bucket predictions to LAVA and 73–75% of incorrect-bucket ones, while the original gate routes essentially all of both. The fused estimate, rather than the gate, appears to absorb most of the damage from wrong predictions. NILAS-Bayes is nominally the strongest fused variant at low accuracy; that comparison would need its own confirmatory test. Hidden error-bucket labels are reconstructed only by the analysis and never reach the scheduler.

10 Robustness and Sensitivity

The robustness suites use fresh seeds 2000–2049 (n = 50). Each variant changes one assumption: the severity of incorrect predictions, a misspecified error spread in the fusion model, a misspecified prior or the load. The first row of Tables 10.1 and 10.2 is the unmodified protocol on the same seeds. These comparisons are descriptive and not corrected for multiplicity.

Table 10.1. VM-CA-Bayes minus LAVA, pp, mean and [95% CI]. Green: interval above zero; red: interval below zero.

Variant	25%	50%	75%	85%
Protocol, same seeds	+0.225 [+0.141, +0.309]	+0.185 [+0.135, +0.235]	+0.113 [+0.043, +0.182]	+0.127 [+0.046, +0.208]
Incorrect-bucket σ = 1	+0.024 [−0.045, +0.093]	+0.102 [+0.013, +0.191]	+0.025 [−0.039, +0.089]	+0.093 [+0.025, +0.162]
Incorrect-bucket σ = 0.5	+0.070 [−0.009, +0.149]	+0.062 [−0.011, +0.135]	+0.037 [−0.037, +0.112]	+0.092 [+0.026, +0.159]
Fusion assumes σ = 3, true σ = 1	+0.083 [+0.008, +0.157]	+0.074 [+0.003, +0.146]	+0.002 [−0.073, +0.077]	+0.066 [0.000, +0.133]
Fusion assumes σ = 1, true σ = 3	+0.168 [+0.101, +0.236]	+0.173 [+0.116, +0.231]	+0.109 [+0.020, +0.199]	+0.069 [−0.011, +0.148]
Prior means × 0.5	+0.267 [+0.171, +0.362]	+0.135 [+0.061, +0.210]	−0.033 [−0.111, +0.045]	−0.116 [−0.197, −0.034]
Prior means × 2	+0.223 [+0.142, +0.303]	+0.215 [+0.135, +0.295]	+0.149 [+0.069, +0.229]	+0.112 [+0.043, +0.181]
Arrival rate × 2 (load × 2)	+0.476 [+0.374, +0.577]	+0.271 [+0.170, +0.372]	+0.177 [+0.088, +0.266]	+0.101 [−0.012, +0.215]

Table 10.2. VM-CA-Bayes minus Best-Fit under the same variants.

Variant	25%	50%	75%	85%
Protocol, same seeds	+0.039 [−0.037, +0.115]	+0.127 [+0.049, +0.204]	+0.280 [+0.204, +0.357]	+0.434 [+0.356, +0.513]
Incorrect-bucket σ = 1	+0.062 [−0.008, +0.131]	+0.196 [+0.119, +0.273]	+0.336 [+0.255, +0.417]	+0.464 [+0.385, +0.542]
Incorrect-bucket σ = 0.5	+0.189 [+0.111, +0.267]	+0.285 [+0.205, +0.365]	+0.389 [+0.315, +0.462]	+0.478 [+0.404, +0.553]
Fusion assumes σ = 3, true σ = 1	+0.120 [+0.047, +0.193]	+0.169 [+0.098, +0.239]	+0.313 [+0.241, +0.385]	+0.437 [+0.360, +0.513]
Fusion assumes σ = 1, true σ = 3	−0.017 [−0.086, +0.051]	+0.115 [+0.031, +0.199]	+0.277 [+0.193, +0.361]	+0.376 [+0.302, +0.450]
Prior means × 0.5	+0.081 [−0.015, +0.176]	+0.077 [−0.012, +0.166]	+0.135 [+0.065, +0.204]	+0.191 [+0.111, +0.272]

Variant	25%	50%	75%	85%
Prior means × 2	+0.037 [−0.044, +0.118]	+0.157 [+0.078, +0.236]	+0.317 [+0.262, +0.372]	+0.419 [+0.343, +0.495]
Arrival rate × 2 (load × 2)	+0.100 [+0.009, +0.192]	+0.262 [+0.145, +0.379]	+0.524 [+0.411, +0.637]	+0.666 [+0.544, +0.789]

On the same seeds the unmodified protocol gives +0.225, +0.185, +0.113, +0.127 pp over LAVA at 25–85% accuracy, all with intervals above zero. Milder errors (σ = 1 or 0.5) make wrong predictions harder to recognize. Gains over LAVA shrink and are significant in only one or two cells, but never significantly negative; gains over Best-Fit remain significant from 50% accuracy upward. A misspecified error spread keeps a positive mean in every cell, with intervals above zero in two or three of four.

Prior means × 2 keeps all four gains over LAVA. This is consistent with overestimation being the cheaper error here: a short VM on a long-lived host costs little, while a long VM on a short-lived host keeps it busy. With halved prior means, correct predictions of long lifetimes look implausible. For a service VM at 85% accuracy whose correct value sits at the 336-hour cap, the posterior mean drops from 251 h under the true prior to 72 h. VM-CA-Bayes is then worse than LAVA at 85% accuracy (−0.116 [−0.197, −0.034] pp; one descriptive cell of 32, unadjusted p = 0.006), while still better than Best-Fit (+0.191 [+0.111, +0.272] pp). This failure motivates learning priors from history and testing systematic underestimation.

Doubling the arrival rate (16 VMs per hour, 5,000 VMs, same 180 hosts) lowers Best-Fit's empty share to 80.7%. LAVA's penalty at 25% accuracy deepens to −0.376 pp, and the fused advantage over LAVA grows at 25–75% accuracy (+0.476, +0.271, +0.177 pp against +0.225, +0.185, +0.113 on the protocol); at 85% it is +0.101 [−0.012, +0.215] pp. An earlier variant with 90 hosts turned out to be a pure rescaling: fewer than 90 hosts are ever busy and ties go to the lowest host index, so every placement equals the 180-host run and only the denominator of pp changes. It is not reported as a load test.

Threshold and split sensitivity

On threshold-sensitivity seeds 200–219 (Table 10.3), thresholds from 0.1 to 0.5 give a positive mean gain over LAVA at every accuracy, with intervals above zero in 12 of 16 cells. Thresholds of 0.7 and above still help at 25% accuracy but give negative means at 50–85%, with intervals below zero in 3 of 9 cells. These thresholds still route 64–80% of arrivals to LAVA at 75–85% accuracy, against 81–85% at τ = 0.3; sending the additional requests to Best-Fit costs more than it saves. The frozen τ = 0.3 was not chosen from these numbers.

Table 10.3. VM-CA-Bayes minus LAVA by threshold, seeds 200–219 (n = 20), pp.

τ	25%	50%	75%	85%
0.1	+0.208 [+0.079, +0.338]	+0.139 [+0.023, +0.256]	+0.109 [−0.024, +0.241]	+0.139 [+0.021, +0.258]
0.2	+0.268 [+0.185, +0.352]	+0.112 [+0.002, +0.222]	+0.135 [+0.002, +0.268]	+0.109 [+0.023, +0.194]
0.3	+0.274 [+0.156, +0.392]	+0.192 [+0.072, +0.312]	+0.130 [−0.021, +0.281]	+0.157 [+0.063, +0.252]
0.5	+0.172 [+0.093, +0.252]	+0.074 [−0.041, +0.189]	+0.039 [−0.076, +0.154]	+0.142 [+0.002, +0.281]
0.7	+0.197 [+0.086, +0.308]	−0.037 [−0.165, +0.090]	−0.125 [−0.240, −0.011]	−0.090 [−0.196, +0.016]
0.8	+0.124 [+0.040, +0.208]	−0.005 [−0.106, +0.096]	−0.113 [−0.256, +0.029]	−0.148 [−0.234, −0.062]
0.9	+0.180 [+0.074, +0.287]	−0.023 [−0.130, +0.084]	−0.094 [−0.228, +0.040]	−0.104 [−0.198, −0.011]

Rerunning the held-out seeds 20–39 with all ten policies reproduces the original-policy columns of Table 7.1 (identical integer decisions; floating-point differences $\leq 3.3\times10^{-16}$) and shows the same pattern for the fused variant (complete archived results).

11 Discussion and Limitations

The fusion prior is the generating profile distribution, the same model the default predictor already uses. In practice it would be estimated from history; Table 10.1 shows the risk of underestimating lifetimes. The scheduler is assumed to know the global accuracy and the error spreads, as the original confidence model also assumes. Both directions of σ misspecification were tested. The σ = 3 incorrect bucket of the protocol spans orders of magnitude and is easy to flag as implausible; milder errors give smaller gains.

Mixture parameters, profile probabilities and VM shapes are inferred, and only the load was varied. About 68% of the evaluation horizon is the drain tail; the arrival-window metric points the same way for the fused predictor (+0.312, +0.207, +0.076 pp over LAVA at 25%, 50% and 85%). Absolute values are not comparable with production results [1]. NILAS outperforms LAVA at most accuracies here (Table 9.2), unlike in [1]; our class,

deadline and tie-breaking rules are reconstructed choices. The Oracle is a reference, not an upper bound: NILAS-Bayes at 100% accuracy (+0.599 pp) exceeds the NILAS Oracle (+0.581 pp). The original code, traces and random streams were not recovered. Disagreement with the surviving report does not falsify it, because critical parameters are unknown.

Only the eight-test primary family is multiplicity-adjusted. The decomposition, the NILAS-Bayes observation and all robustness cells are descriptive. The registration is a self-timestamped local file.

Future evaluation of learned lifetime distributions should combine proper scoring rules [20] with downstream allocation metrics. Uncertainty benchmarks on classification tasks demonstrate deterioration under dataset shift [21], motivating temporal workload splits and drift tests of both VM priors and error models. The current misspecification experiments probe selected assumptions rather than that broader deployment setting.

Within these limits the internal comparisons are strong. Every policy sees identical traces and noisy values, every comparison is paired, admission is equal, and the main claims rest on seeds that were unused when the hypotheses were registered.

12 Conclusion and Reproducibility

Prediction quality determines whether LAVA helps or harms consolidation in this reconstructed setting. Confidence-aware routing does not resolve low-quality predictions: the confidence signal is inverted under the default predictor, the gate is nearly unselective under controlled noise, and host isolation incurs a packing cost. Prior fusion addresses the error in the lifetime estimate without changing the scheduler. It improves on LAVA in 3 of 4 pre-registered tests on 100 fresh seeds; the 75% test remains unconfirmed. The post-hoc decomposition points to the estimate, rather than the gate, as the main source of improvement. Underestimating long lifetimes was the main observed failure mode. Learned priors and production traces are needed to establish how far the result transfers beyond the synthetic workload.

Reproducibility

Python 3.9 or newer and the standard library run every experiment; only the report and slide builders need third-party packages (reportlab and python-pptx). From the project directory:

```
python3 -m unittest discover -s tests -v
python3 run.py --suite all --workers 8 --output results/my_all
python3 run_improvement.py --suite all --workers 8 --output results/improvement
python3 scripts/run_robust_extra.py --suite control --workers 8 # and --suite load
python3 scripts/verify_reproduction.py
python3 scripts/build_report_v8.py
```

Raw per-seed results, paired summaries, manifests with source hashes, the registration file and figures are under results/ and configs/. IMPROVEMENT.md and REPRODUCTION_DIFFERENCES.md (both in Chinese) record every assumption and its status. The rebuilt source should not be cited as the lost original commit 377cbb1.

Artifact and provenance. Historical course materials are archived under *reference/*; the reconstructed code, frozen results and report sources are available in the project repository. These project materials are described separately from the academic bibliography.

References

[1] J. Ling et al. LAVA: Lifetime-Aware VM Allocation with Learned Distributions and Adaptation to Mispredictions. *Proceedings of Machine Learning and Systems*, vol. 7, 2025. Paper. Extended version, Appendix G.1.

[2] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes. Large-scale cluster management at Google with Borg. *EuroSys*, pp. 1-17, 2015. doi:10.1145/2741948.2741964.

[3] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini. Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms. *SOSP*, pp. 153-167, 2017. doi:10.1145/3132747.3132772.

[4] N. Buchbinder, Y. Fairstein, K. Mellou, I. Menache, and J. Naor. Online Virtual Machine Allocation with Lifetime and Load Predictions. *SIGMETRICS*, pp. 9-10, 2021. doi:10.1145/3410220.3456278. Author publication page and full paper.

[5] H. Barbalho et al. Virtual Machine Allocation with Lifetime Predictions. *Proceedings of Machine Learning and Systems*, vol. 5, 2023. Paper.

[6] O. Hadary et al. Protean: VM Allocation Service at Scale. *OSDI*, pp. 845-861, 2020. Paper.

[7] A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin. Bayesian Data Analysis. *Chapman & Hall/CRC*, 3rd ed., 2013. doi:10.1201/b16018.

[8] E. L. Kaplan and P. Meier. Nonparametric Estimation from Incomplete Observations. *Journal of the American Statistical Association*, 53(282):457-481, 1958. doi:10.1080/01621459.1958.10501452.

[9] C. Delimitrou and C. Kozyrakis. Quasar: Resource-Efficient and QoS-Aware Cluster Management. *ASPLOS*, pp. 127-144, 2014. doi:10.1145/2541940.2541941.

[10] A. Tumanov, T. Zhu, J. W. Park, M. A. Kozuch, M. Harchol-Balter, and G. R. Ganger. TetriSched: Global

Rescheduling with Adaptive Plan-Ahead in Dynamic Heterogeneous Clusters. *EuroSys*, Article 35, pp. 1-16, 2016. doi:10.1145/2901318.2901355.

[11] J. W. Park, A. Tumanov, A. Jiang, M. A. Kozuch, and G. R. Ganger. 3Sigma: Distribution-based cluster scheduling for runtime uncertainty. *EuroSys*, pp. 1-17, 2018. doi:10.1145/3190508.3190515.

[12] M. Maas, D. G. Andersen, M. Isard, M. M. Javanmard, K. S. McKinley, and C. Raffel. Learning-based Memory Allocation for C++ Server Workloads. *ASPLOS*, pp. 541-556, 2020. doi:10.1145/3373376.3378525.

[13] D. R. Cox. Regression Models and Life-Tables. *Journal of the Royal Statistical Society, Series B*, 34(2):187-202, 1972. doi:10.1111/j.2517-6161.1972.tb00899.x.

[14] Student. The Probable Error of a Mean. *Biometrika*, 6(1):1-25, 1908. doi:10.1093/biomet/6.1.1.

[15] S. Holm. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6(2):65-70, 1979. Paper.

[16] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor. The Preregistration Revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600-2606, 2018. doi:10.1073/pnas.1708274114.

[17] R. Kumar, M. Purohit, and Z. Svitkina. Improving Online Algorithms via ML Predictions. *Advances in Neural Information Processing Systems*, 31:9661-9670, 2018. Paper.

[18] M. Liu and X. Tang. Tight Bounds for Dynamic Bin Packing with Predictions. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 8(3), Article 47, pp. 1-28, 2024. doi:10.1145/3700437.

[19] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On Calibration of Modern Neural Networks. *ICML, Proceedings of Machine Learning Research*, 70:1321-1330, 2017. Paper.

[20] T. Gneiting and A. E. Raftery. Strictly Proper Scoring Rules, Prediction, and Estimation. *Journal of the American Statistical Association*, 102(477):359-378, 2007. doi:10.1198/016214506000001437.

[21] Y. Ovadia et al. Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift. *Advances in Neural Information Processing Systems*, 32:13991-14002, 2019. Paper.